Universidad Nacional de San Agustín de Arequipa

Escuela de Ingeniería de Sistemas

Sesión 6**: Pathfinding**

I. OBJETIVOS

- Conocer los algoritmos de búsqueda de caminos.
- Utilizar los algoritmos para implementarlo en un video juego.

II. TEMAS A TRATAR

- Búsqueda de caminos
- Algoritmos de pathfinding

III. MARCO TEORICO

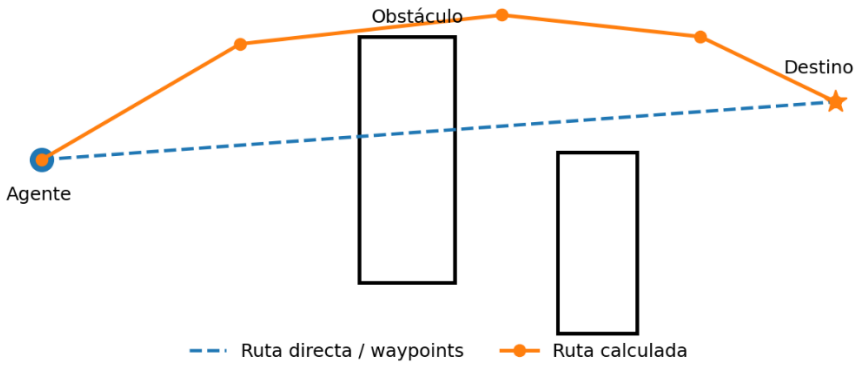
- **Pathfinding**

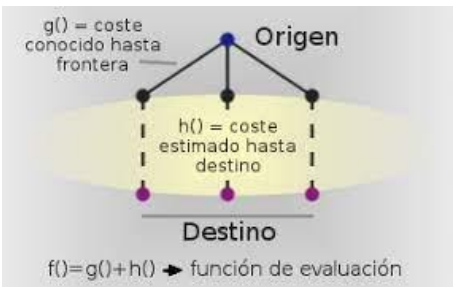
  Pathfinding provee una base importante para gran parte de los sistemas de navegación, se utiliza en áreas de robótica, y especialmente en videojuegos.

  El objetivo principal es encontrar el mejor camino posible de un punto inicial hasta un punto final en representaciones del entorno llamadas mapas de malla. Para cada personaje, la IA debe poder calcular una ruta adecuada a través del nivel del juego para llegar desde su posición hasta su objetivo. Lo ideal es saber si la ruta es la más corta o la más rápida.

  Esto es la búsqueda de caminos (pathfinding), a veces llamada planificación de rutas. La gran mayoría de los juegos utiliza algoritmos de búsqueda de rutas basados en A*. Es un algoritmo eficiente y fácil de implementar; A* requiere que el nivel del juego esté representado como un grafo no dirigido ponderado.

Waypoints vs. Pathfinding: ruta prefijada y ruta calculada





Para cada nodo considera:

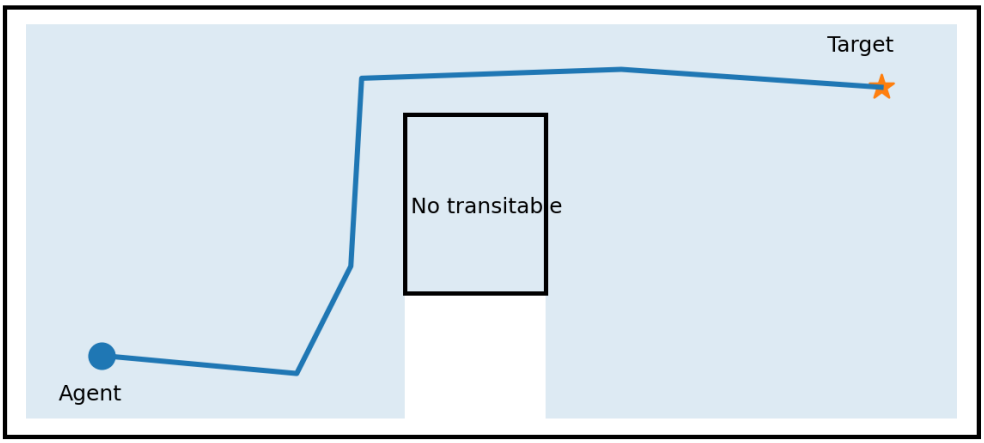
- g(n): costo real acumulado desde el nodo inicial hasta n.
- h(n): estimación heurística desde n hasta la meta.
- f(n) = g(n) + h(n): valor utilizado para priorizar qué nodo explorar.

Idea didáctica: un waypoint responde a la pregunta "¿qué puntos debo visitar?". Pathfinding responde a la pregunta: "¿por dónde puedo llegar al objetivo sin atravesar zonas intransitables?".

AI Navigation in Unity

El sistema de navegación permite crear personajes que se mueven de forma inteligente por el mundo del juego. Estos personajes utilizan mallas de navegación generadas automáticamente a partir de la geometría de la escena. Los obstáculos dinámicos permiten modificar la navegación de los personajes en tiempo de ejecución, mientras que los enlaces de NavMesh permiten definir acciones específicas, como abrir puertas, saltar sobre huecos o descender desde un saliente.

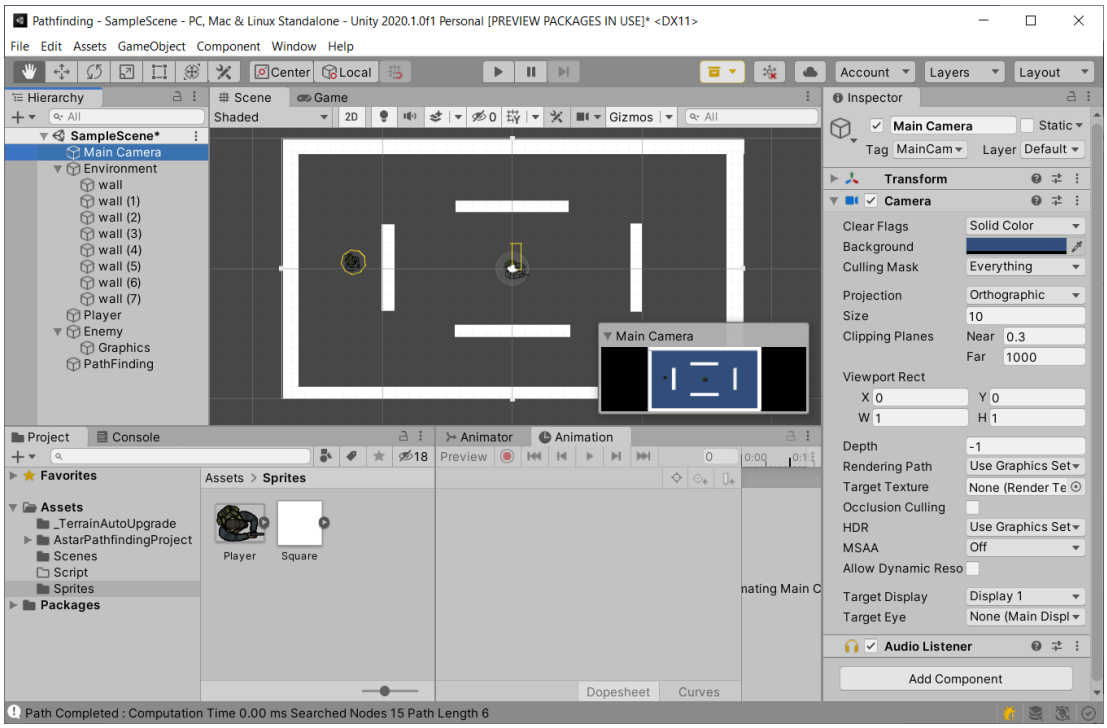
NavMesh: superficie navegable + obstáculos + ruta del agente



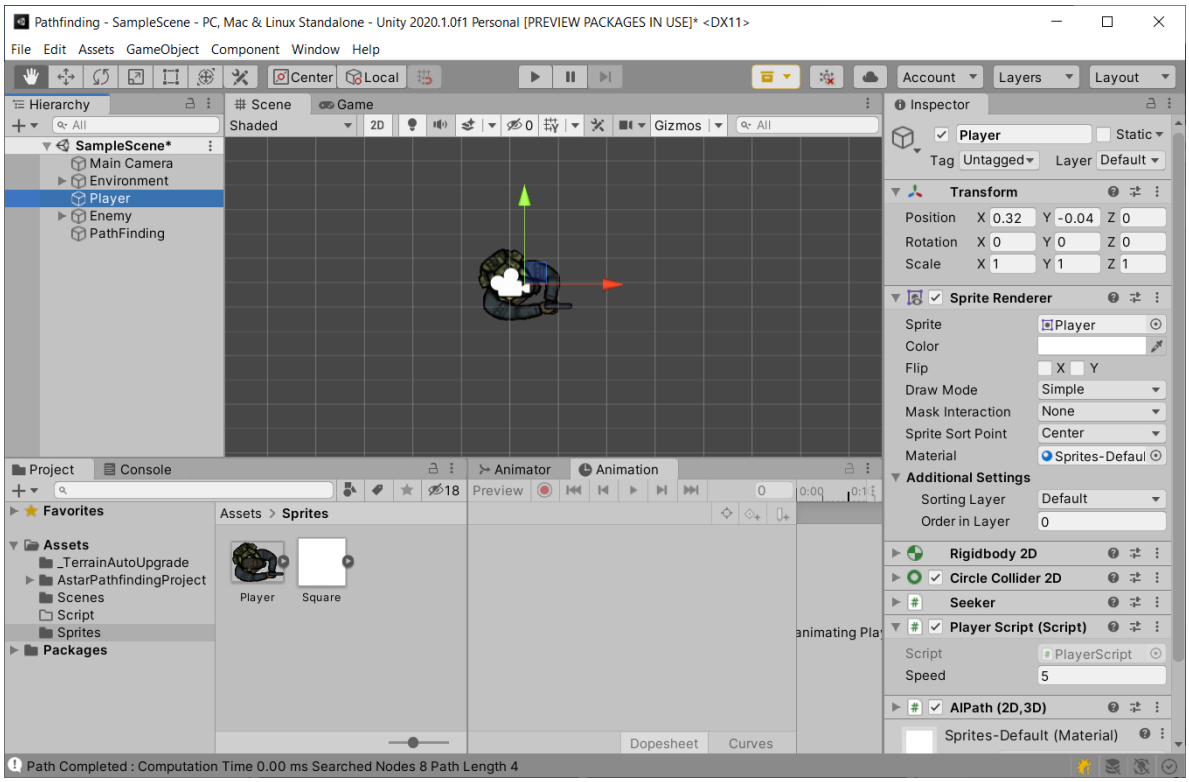
IV. ACTIVIDADES

Desarrollo del juego con pathfinding:

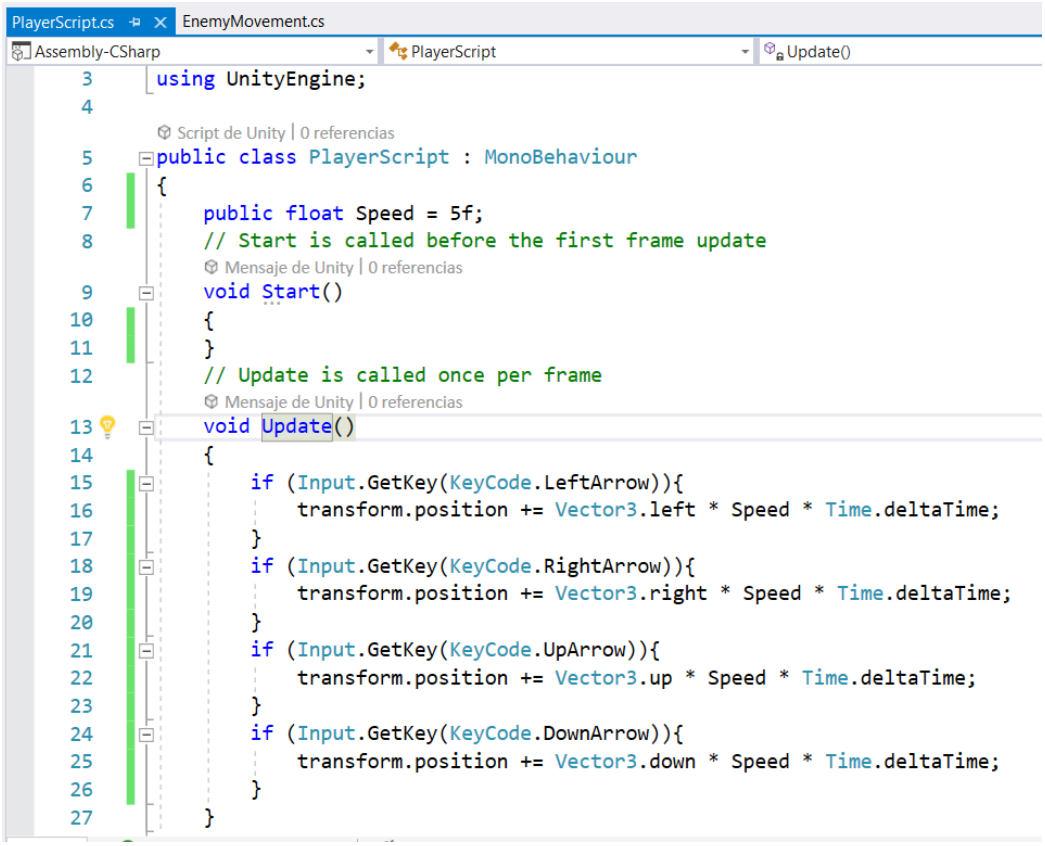
1. En Unity cree un nuevo proyecto 2D y diseñe el siguiente escenario



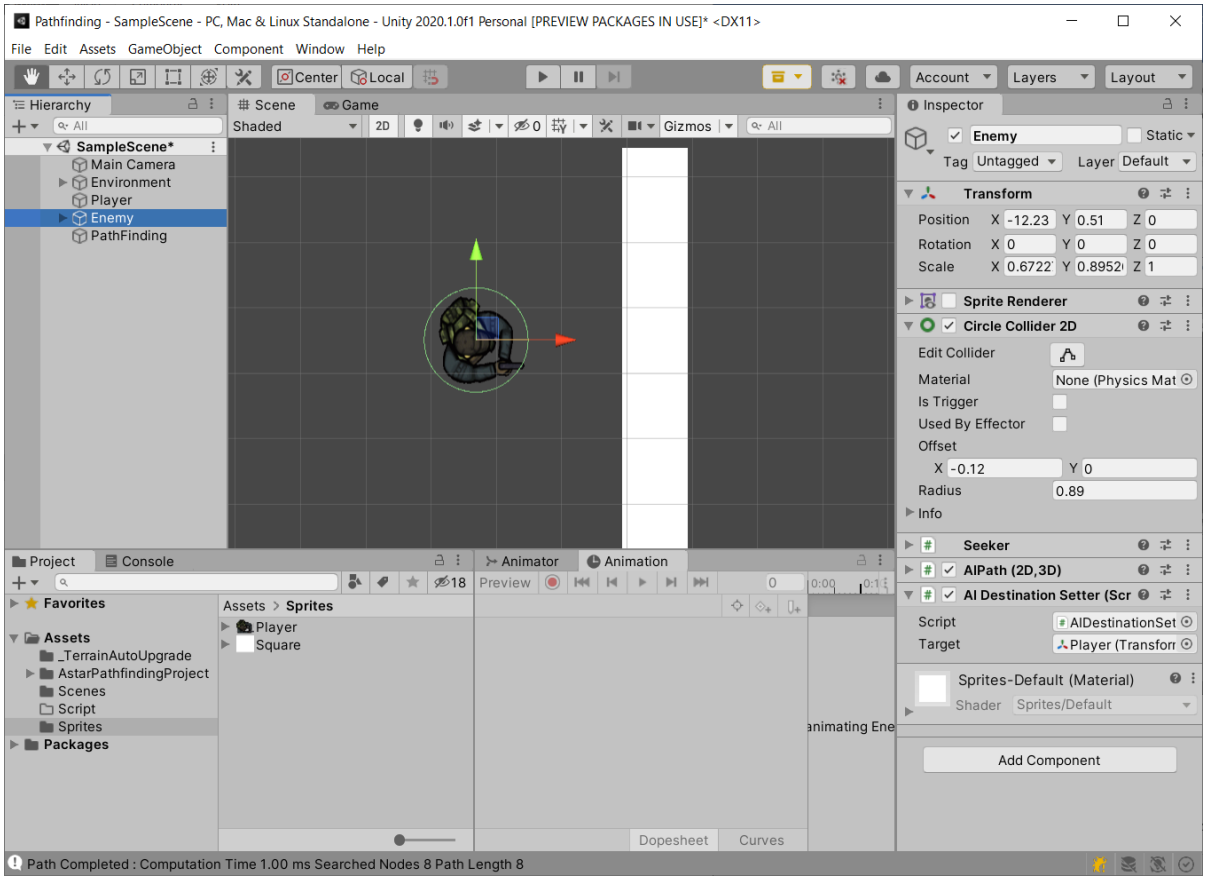
2. Descargue al personaje del siguiente enlace:
   https://drive.google.com/file/d/1Epzoo14fYCxe6VsyiHPyt3BMpFhn3_p-/view

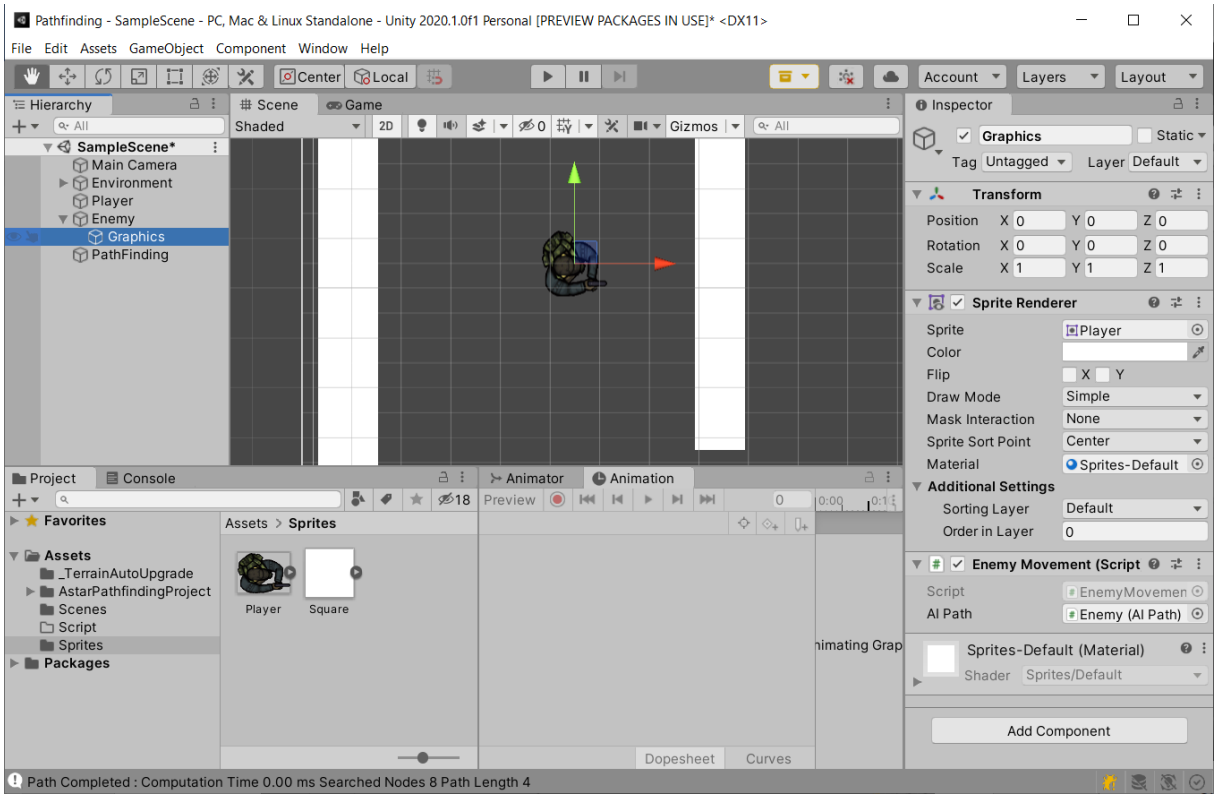


3. Agregue el Script (PlayerScript) para controlar el movimiento del Player

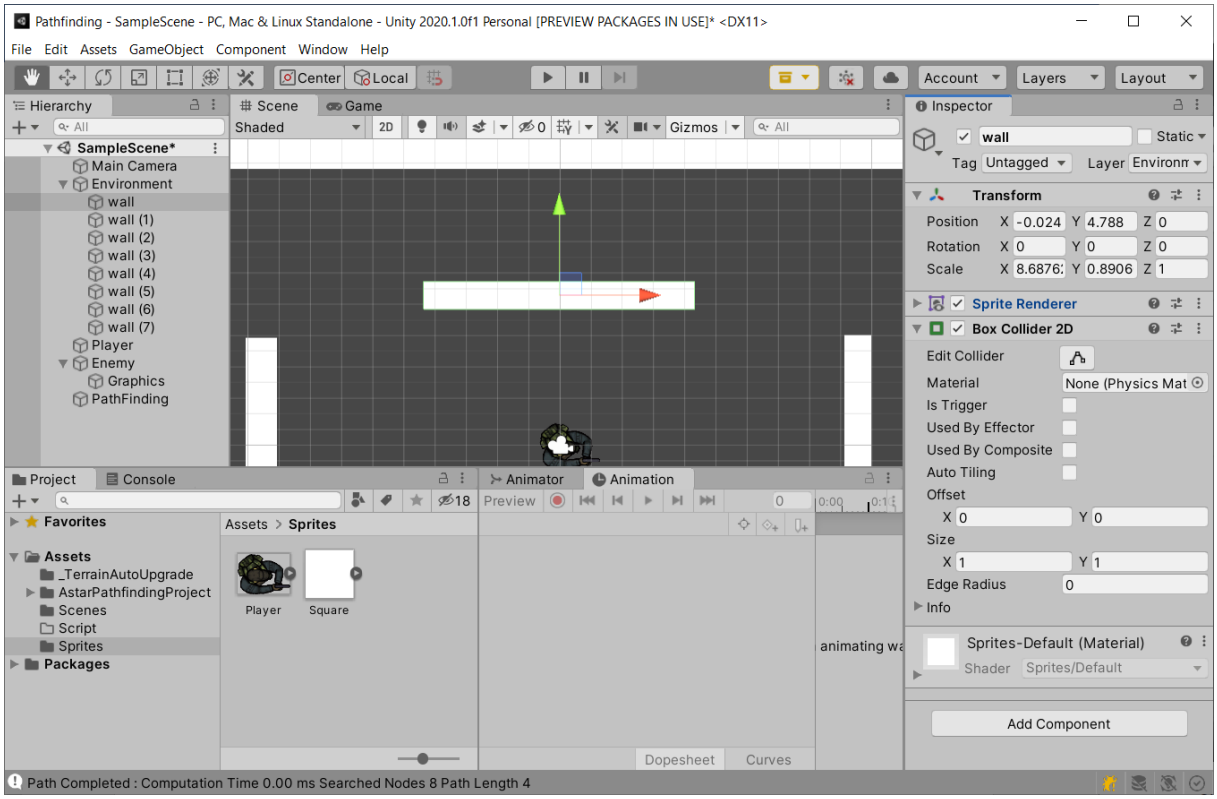


4. Agregue el sprite Enemy al escenario





5. Configurar las paredes (wall) con los componentes Collider



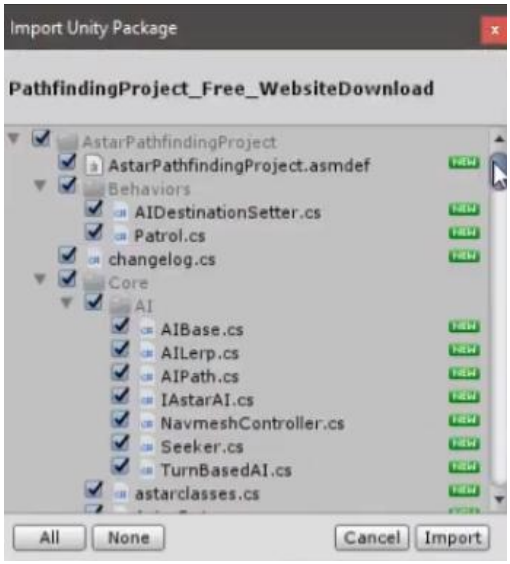
6. Para el movimiento del Enemy se utilizarán los conceptos de Pathfinding. para que pueda encontrar el camino más corto disponible para alcanzar su objetivo (Player), por lo que se usará el algoritmo A* Pathfinding que se puede descargar de forma gratuita utilizando el siguiente enlace:

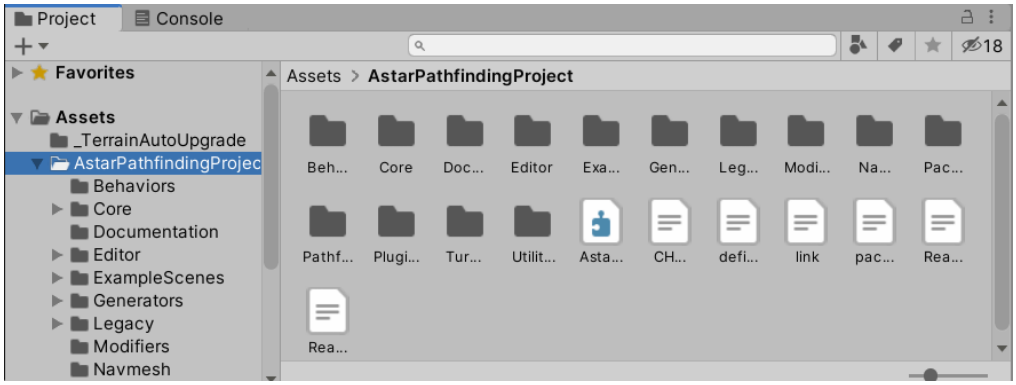
   https://arongranberg.com/astar/download

   Free Version

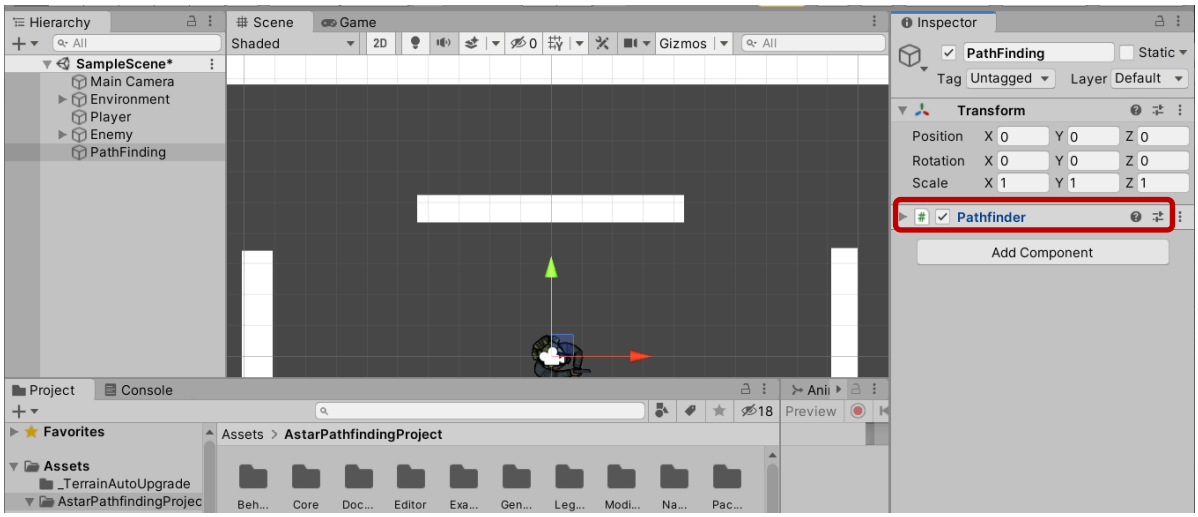
   Free version of the project. Slightly more limited than the pro version, but still very powerful.
   Go ahead and try it out!
   Includes several example scenes showing how to use the system.

7. Desempaquete el archivo y luego impórtelo en su Proyecto.

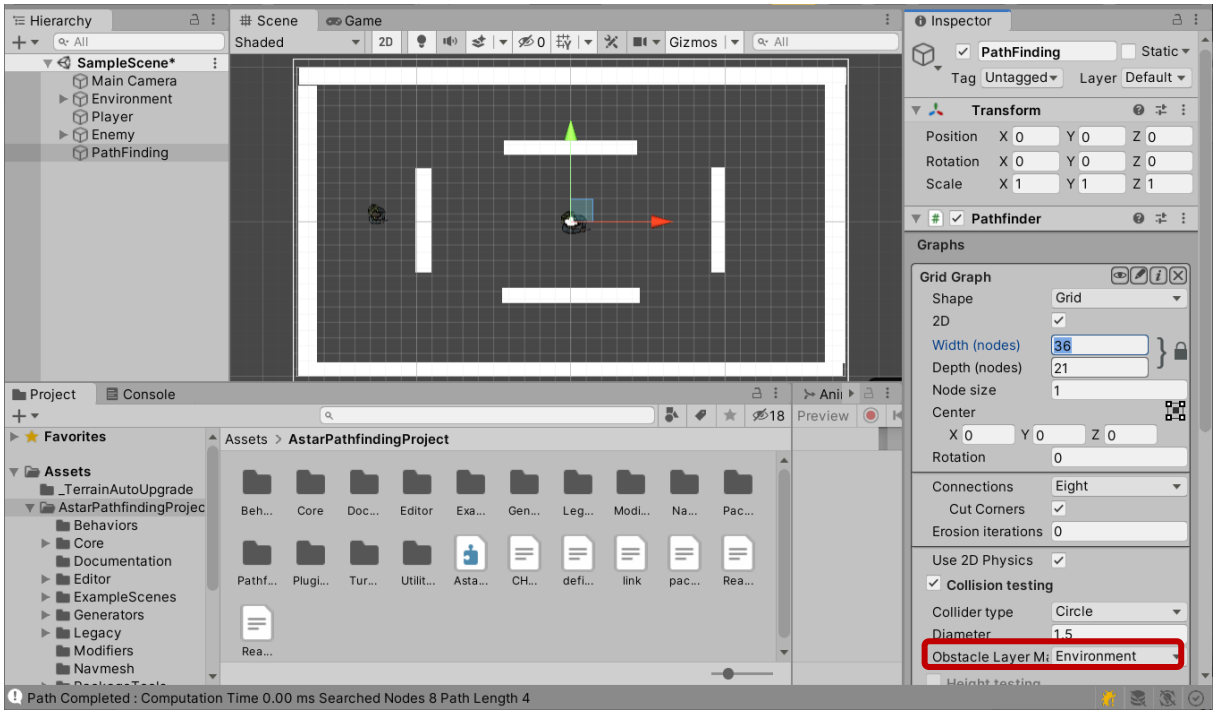




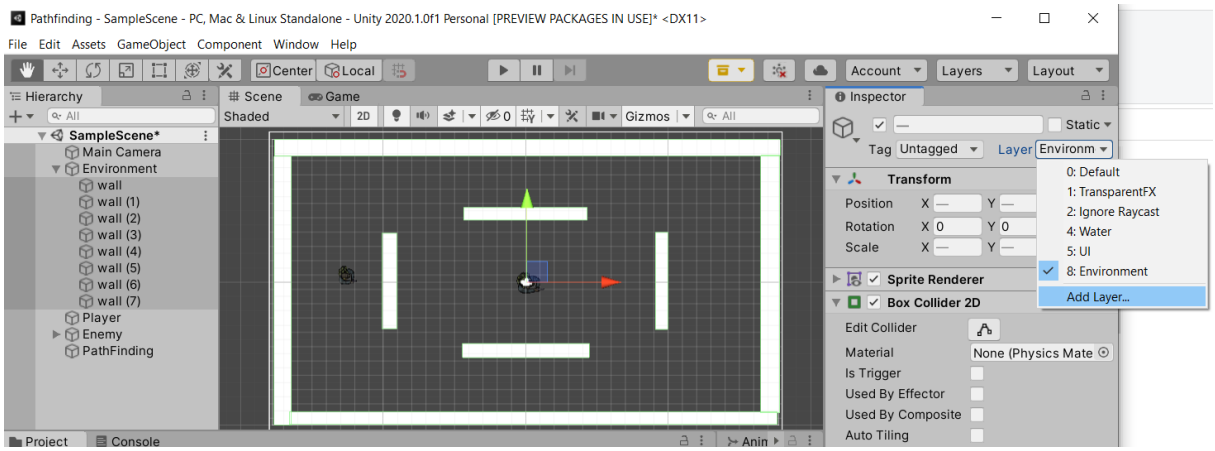
8. Agregue un nuevo GameObject (Create Empty) llamado PathFinding y añada el componente **PathFinder**.

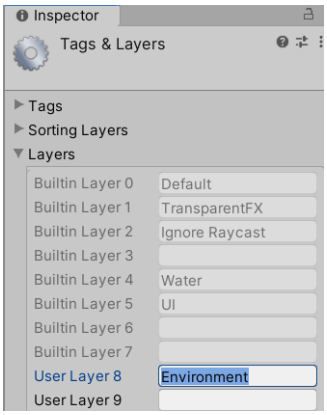


9. Haga click en la opción *Graphs / Grid Graph* para configurar el área 2D del pathfinding. Así como **Use Physic 2D**.

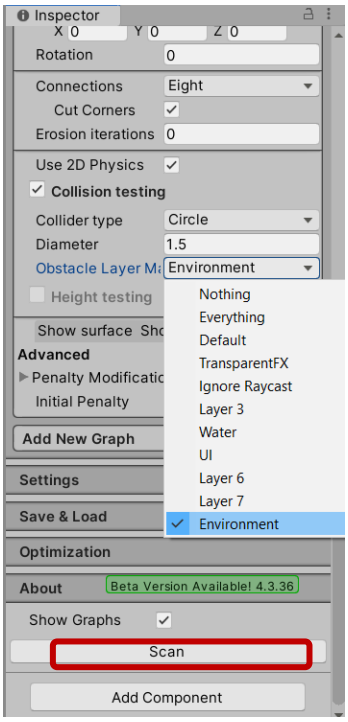


10. Para la opción **Obstacle Layer Mask.** Se seleccionará todas las paredes (wall) y se agregará una nueva capa (Add Layer…) con todos ellos con el nombre de *User Layer 8*: **Environment**.

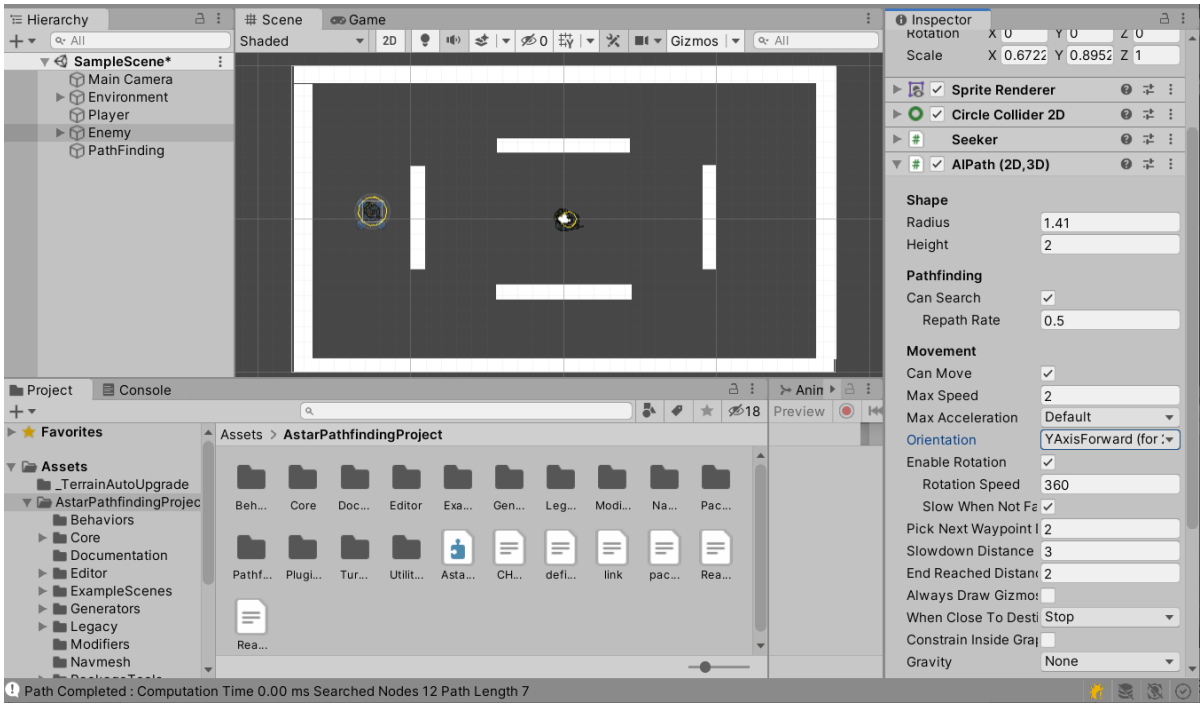




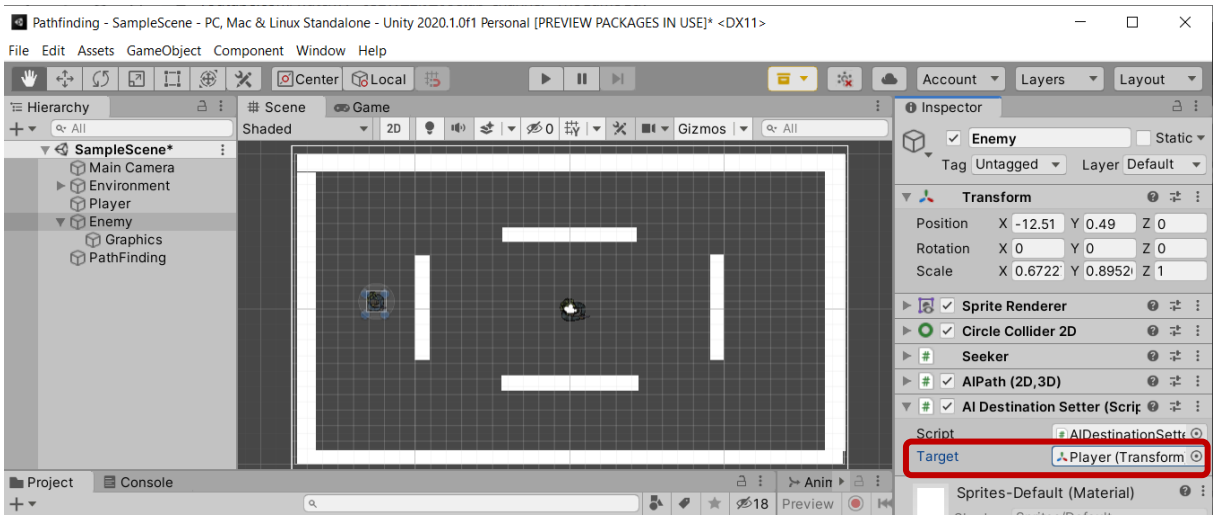
11. Presione el botón **Scan** para actualizarlo.



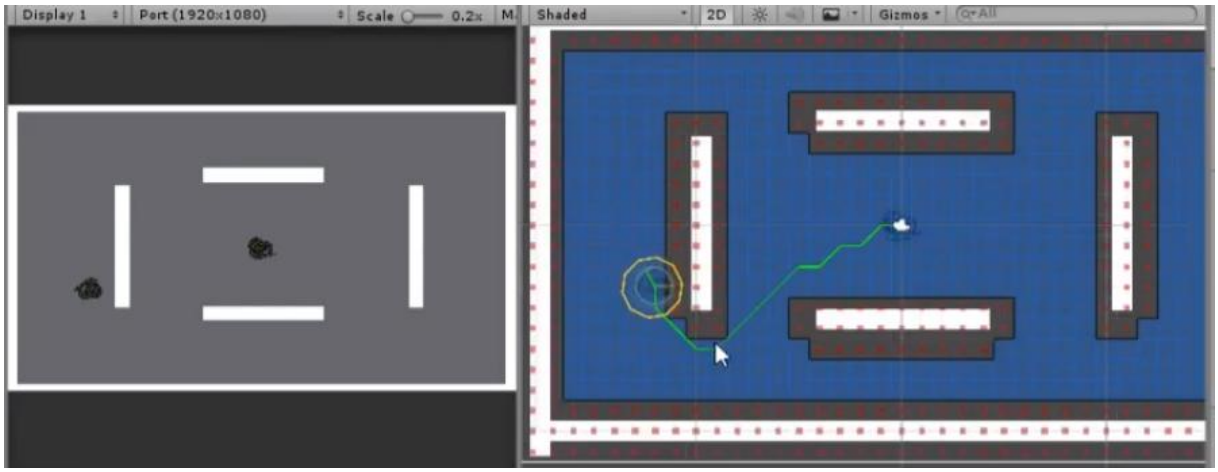
12. Agregue al sprite *Enemy* un componente **AI Path (2D 3D)**



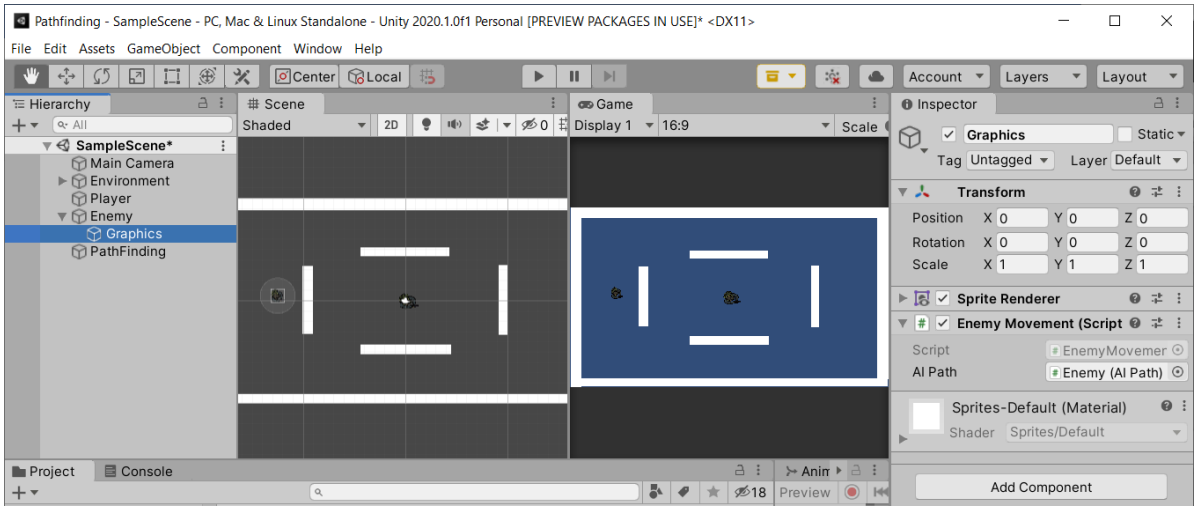
13. Agregue al sprite Enemy un componente **AI Destination Setter (Script)**, para poner como *Target* al **Player.**



14. Al ejecutar podrá ver la ruta que encontró el enemigo hacia su objetivo (Player)



15. Para darle la direccionalidad al Enemy, agregue un script (**EnemyMovement**) al elemento **Graphics.**



16. Agregue el siguiente código al script **EnemyMovement:**

```
layerScript.cs   EnemyMovement.cs
Assembly-CSharp   EnemyMovement   aIPath
1   using System.Collections;
2   using System.Collections.Generic;
3   using UnityEngine;
4   using Pathfinding;
5
    Script de Unity | 0 referencias
6   public class EnemyMovement : MonoBehaviour
7   {
8       public AIPath aIPath;
9       Vector2 direction;
10      // Start is called before the first frame update
        Mensaje de Unity | 0 referencias
11      void Start()
12      {
13      }
14      // Update is called once per frame
        Mensaje de Unity | 0 referencias
15      void Update()
16      {
17          faceVelocity();
18      }
        1 referencia
19      void faceVelocity()
20      {
21          direction = aIPath.desiredVelocity;
22          transform.right = direction;
23      }
24  }
```

17. Ejecute el juego

V. Ejercicios

1. Al App desarrollado, agregue otras paredes como obstáculos, así como otro Enemy en la escena.
2. Implemente el ejemplo del siguiente enlace:
   The A* Algorithm - Unity Learn
3. Haga un informe de la solución a los ejercicios planteados.

VI. Cuestionario

1. ¿Cómo funciona el algoritmo de pathfinding A*?
2. ¿Qué componentes existen para apoyar la implementación de pathfinding en videojuegos?

VII. Bibliografía y referencias

1. Millington, I., & Funge, J. (2019). Artificial intelligence for games. Third Ed. CRC Press.
2. https://www.youtube.com/watch?v=63zWZ2rJZ68&ab_channel=TheGameGuy
3. Unity Technologies. AI Navigation package (com.unity.ai.navigation).
4. The A* Algorithm - Unity Learn